

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering ASSESSMENT TOOL 1 – Coding Challenge Task

Course Code: CSA0604 **Course Title:** Design and Analysis of Algorithms

CO Assessed: CO4: Articulation (BL4, BL5) **Assessment:** Assessment Tool 1 – Coding Challenge Task

Weightage: 40% of CIA **Total Marks:** 100

CO4 Statement: Solve optimization problems using dynamic programming and greedy strategies.

Student Name: G Hari Prasad **Reg. No.:** 192525273

Section / Batch: B **Date:** 19-08-2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given questions.
- Write clear code for the given problem.
- Analyze the Time complexity and Space complexity of your code using dynamic programming and greedy strategies.

Question Type	Focus Area	Questions
Coding Challenge Task	Focus on Code and analyze optimization problems using dynamic programming and greedy strategies	Q36

SECTION A – Coding Challenge Task

Code of Conduct

I G Hari Prasad / 192525273 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: G Hari Prasad

RUBRICS FOR CALCULATION

Coding Challenge Task

Criteria	Wt.	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Marks Evaluation:

Criteria	Wt.	Level 4	Level 3	Level 2	Level 1
Problem Understanding	20%				
Algorithm	25%				
Code Implementation	20%				
Competitive Performance	20%				
Test Case Handling	15%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature:	Signature:	Signature:
Date:	Date:	Date:

Q36. Unique Paths in Grid — Solution

Name: G Hari Prasad Reg. No.: 192525273

1. Problem Understanding (20%)

A robot starts at the top-left corner of an $m \times n$ grid and must reach the bottom-right corner, moving only right or down at each step. The task is to count the total number of distinct paths.

Constraints / edge cases considered: a grid with only 1 row or 1 column (only 1 possible path), square vs. rectangular grids, and large grids (m, n up to ~ 100 or more) where a purely recursive approach would be too slow due to repeated recomputation.

Sample Input: $m = 3, n = 3$ **Expected Output:** Paths = 6

2. Algorithm (25%)

A brute-force recursive solution that tries "move right" and "move down" at every cell has exponential time complexity $O(2^{(m+n)})$, which is not competitive for large grids since the same subgrid is recomputed many times (overlapping subproblems).

Instead, we use bottom-up Dynamic Programming: let $dp[i][j]$ be the number of unique paths to reach cell (i, j) from $(0, 0)$.

Recurrence: $dp[i][j] = dp[i-1][j] + dp[i][j-1]$, since the robot can only arrive at (i, j) from above or from the left.

Base case: $dp[0][j] = 1$ for all j , and $dp[i][0] = 1$ for all i . Answer: $dp[m-1][n-1]$.

This can be optimized to a 1-D array of size n , updated in place row by row, reducing space from $O(m \cdot n)$ to $O(n)$. Every cell is computed exactly once from previously computed values — no repeated work.

3. Code Implementation (20%)

```
def uniquePaths(m, n):
    # dp represents one row; initialize first row as all 1s
    dp = [1] * n
    for i in range(1, m):
        for j in range(1, n):
            dp[j] = dp[j] + dp[j - 1]
    return dp[n - 1]

if __name__ == "__main__":
    m, n = 3, 3
    print(f"Grid: {m} x {n}")
    result = uniquePaths(m, n)
    print("Paths =", result)
```

Execution Output:

```
$ python3 unique_paths.py
Grid: 3 x 3
Paths = 6
```

4. Competitive Performance (20%)

Metric	Value
Time Complexity	$O(m \cdot n)$ — every cell visited exactly once
Space Complexity	$O(n)$ — 1-D dp array (row-reused, in-place)
Auxiliary Space	$O(1)$ extra beyond the dp array itself

This approach comfortably scales to grids with m, n up to 10^3 – 10^4 within typical time limits, avoiding the exponential blow-up of brute-force recursion and the numerical overhead of large factorial-based combinatorics ($C(m+n-2, m-1)$) for very large inputs.

5. Test Case Handling (15%)

#	Input (m, n)	Expected Output	Result
1	(3, 3)	6	Pass
2	(1, 5)	1	Pass
3	(7, 3)	28	Pass

4	(1, 1)	1	Pass
---	--------	---	------

The program correctly outputs Paths = 6 for the sample 3×3 grid, matching the expected output.